

Files and directories



A **file system** is like the organization system your computer uses to store and find data on a disk (like, HDD, SSD, or USB drive).

Without a file system, your disk would just be a huge sequence of bytes, and the computer wouldn't know,



- * Where one file ends and another begins?
- * What file names are?
- * How to read or write data safely.

So, the file system gives structure and order to storage.

Goal: How should the OS manage a persistent device? What are the APIs? What are the important aspects of the implementation?

What is a **file**? → a **linear array of bytes**, **stored persistently**.
↳ **identified with file name** (human readable) and a **an OS-level identifier called inode number**.

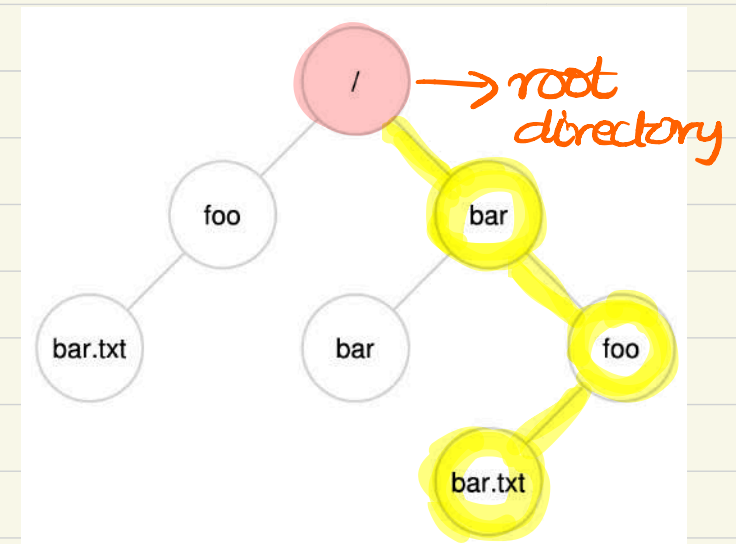
Note that **inode numbers** are unique (at a given time) within a file system.

What is a directory? **contains other sub-directories and files, along with their inode numbers.**

- Stored like a file, whose contents are file name-to-**inode mappings**.
eg:- let "foo" be a file in a directory having the inode number "10"
then the directory would have a corresponding entry ("foo", "10")

Directory tree

- Files and directories are arranged in a tree, starting with root ("/").
we can refer to any file by **absolute path name**. eg:- /foo/bar.txt, /bar/foo/bar.txt, etc.



The file system interface

Basic operations are as follows:

(1) creating files → using the `open()` system call.

eg:- The `fcntl` code is used to open or create a file named `foo`.

flags that tell OS how you want to open the file.

create the file if it doesn't exist open it for writing only if the file already exists erase its contents

```
int fd = open("foo", O_CREAT | O_WRONLY | O_TRUNC,  
              S_IRUSR | S_IWUSR);
```

read and write permissions for the owner.

→ if successful, `open()` returns the file descriptor, a small integer (like 3 or 4) that represents the opened file.

- once a file is opened, the file descriptor would be used to read or write or any operations on the file. eg:- `write(fd, "hello\n", 6);`

What happens internally?

- the system call goes to the kernel
- kernel look up "foo" in the current directory
- if the file doesn't exist and `O_CREAT` is set:
 - creates a new inode and allocates it on disk
- if it exists and `O_TRUNC` is set
 - sets its size to 0 (empties it)
- kernel sets up an open file table entry and returns a descriptor which acts as a handle to the file.

(2) Reading and writing files

↳ uses `read()/write()` system calls.

How to read an existing file?

```
prompt> echo hello > foo
prompt> cat foo
hello
prompt>
```

the shell re-directs `stdout` to the open file descriptor for `foo`.

- after this, a new file named `foo` now exists containing `hello`.

- the `cat` program opens the file `foo` for reading
- it then reads the file contents and writes them to `stdout` (screen)

The following `strace` output shows exactly how the `cat` command interacts with the file system through system calls.

```
prompt> strace cat foo
```

```
...
```

```
open("foo", O_RDONLY|O_LARGEFILE)
```

```
= 3
```

```
read(3, "hello\n", 4096)
```

```
= 6
```

```
write(1, "hello\n", 6)
```

```
= 6
```

```
hello
```

```
read(3, "", 4096)
```

```
= 0
```

```
close(3)
```

```
= 0
```

```
...
```

```
prompt>
```

open the file `foo` and returns `fd=3`

`cat` asks kernel to read upto 4096 bytes from `fd 3`. The kernel returns

"hello\n" → 6 bytes in total

= 6 ⇒ 6 bytes were actually read

writes same 6 bytes to `fd 1`.

= 6 ⇒ write was successful.

tries to read again. The kernel returns 0, meaning end of file (EOF).

⇒ no more data left to read.

↳ closes `fd 3`
= 0 ⇒ success

At the lowest level, **reading a file** happens through the system call

read(fd, buffer, size);
or write

- reading and writing happens sequentially by default.
i.e., successive read/write calls fetch from current offset.

System Calls	Return Code	Current Offset
fd = open("file", O_RDONLY);	3	0
read(fd, buffer, 100);	100	100
read(fd, buffer, 100);	100	200
read(fd, buffer, 100);	100	300
read(fd, buffer, 100);	0	300
close(fd);	0	-

→ opens a file (of size 300 bytes) and reads it by calling read() repeatedly.

→ trace a process that opens same file twice
uses a read() to each of them.

Here, two fd of the same file are allocated and they correspond to different entries in OFT.

(each current offset is updated independently).

Open File Table		OFT[10]	OFT[11]
System Calls	Return Code	Current Offset	Current Offset
fd1 = open("file", O_RDONLY);	3	0	-
fd2 = open("file", O_RDONLY);	4	0	0
read(fd1, buffer1, 100);	100	100	0
read(fd2, buffer2, 100);	100	100	100
close(fd1);	0	-	100
close(fd2);	0	-	-

How to read or write to a specific offset within a file?

→ using **lseek()** system call.

positions the file offset to a particular location within the file.

```
off_t lseek(int filides, off_t offset, int whence);
```

file descriptor

from where to
measure?

If whence is **SEEK_SET**, the offset is set to offset bytes.

If whence is **SEEK_CUR**, the offset is set to its current location plus offset bytes.

If whence is **SEEK_END**, the offset is set to the size of the file plus offset bytes.

On the foll. example **lseek()** first sets the current offset to 200. Then subsequent **read()** then reads the next 50 bytes and updates current offset.

System Calls	Return Code	Current Offset
fd = open("file", O_RDONLY);	3	0
lseek(fd, 200, SEEK_SET);	200	200
read(fd, buffer, 50);	50	250
close(fd);	0	—

Note that the following describes an open file inside the kernel

```
struct file {
```

```
    int ref; → how many fds currently refer to this struct file.
```

```
    char readable;
```

```
    char writable;
```

```
    struct inode *ip;
```

```
    uint off;
```

```
}; current file offset
```

→ pointer to the inode structure

it contains, metadata like

- file size, type (regular file, directory, etc.)
diskblock addresses, permissions, timestamps etc.

The following defines a global table of open files, called the file table (ftable).

Every time any process in the system calls `open()`, the kernel allocates one entry from this table to represent that open file.

```
struct {
```

```
    struct spinlock lock;
```

```
    struct file file[NFILE];
```

```
} ftable;
```

→ ensures that only one CPU can modify the file table at a time

→ each entry represents one open file in the whole system, shared among processes.

Shared file table entries: fork() and dup()

Do the parent and child share the same offset, or have separate copies?

```
int main(int argc, char *argv[]) {
```

```
    int fd = open("file.txt", O_RDONLY);
```

```
    assert(fd >= 0);
```

```
    int rc = fork();
```

```
    if (rc == 0) {
```

```
        rc = lseek(fd, 10, SEEK_SET);
```

```
        printf("child: offset %d\n", rc);
```

```
    } else if (rc > 0) {
```

```
        (void) wait(NULL);
```

```
        printf("parent: offset %d\n",
```

```
              (int) lseek(fd, 0, SEEK_CUR));
```

```
    }
```

```
    return 0;
```

```
}
```

→ opens file.txt for reading and returns a fd.

→ creates a child process that copies the same FD table of the parent

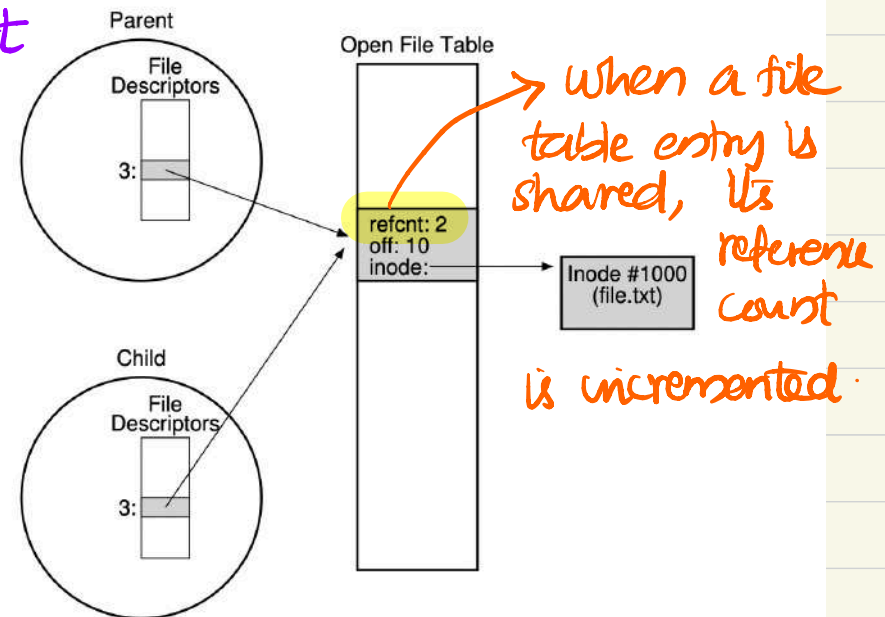
sets the offset in the shared struct file to 10 bytes

ask what is the current offset right now!

→ waits until the child is finished.

Expected output:

```
prompt> ./fork-seeek
child: offset 10
parent: offset 10
prompt>
```



Advantage of sharing open file table

↳ if you create a # processes that are co-operatively working on a task, they can write to the same output file without an extra co-ordination.

The `dup()` system call → allows a process to create a new file descriptor that refers to the same underlying open file as an existing descriptor.

```
int main(int argc, char *argv[]) {
    int fd = open("README", O_RDONLY);
    assert(fd >= 0);
    int fd2 = dup(fd);
    // now fd and fd2 can be used interchangeably
    return 0;
}
```

Exercise

How `dup` (or its variants) are used in output re-direction.

writing immediately with `fsync()` → force all dirty (i.e., not yet written) data to disk, for the file referred to by the specified fd.

```
int fd = open("foo", O_CREAT|O_WRONLY|O_TRUNC,
              S_IRUSR|S_IWUSR);
```

```
assert(fd > -1);
```

```
int rc = write(fd, buffer, size);
```

```
assert(rc == size);
```

```
rc = fsync(fd);
```

```
assert(rc == 0);
```

verify `open()` succeeded
→ may not hit disk yet!
ensure the entire buffer was written
ensure `fsync()` succeeded

Exercise.

Other operations: Renaming a file, deleting a file, get statistics of a file, etc.

Operations on directories

- Directories can also be accessed like files
 - operations like create, open, read, close etc..
- for eg- the **ls** program opens and reads all directory entries.
 - directory entry contains file name, inode number, type of file (file/directory), etc.

```
int main(int argc, char *argv[]) {  
    DIR *dp = opendir(".");  
    assert(dp != NULL);  
    struct dirent *d;  
    while ((d = readdir(dp)) != NULL) {  
        printf("%lu %s\n", (unsigned long) d->d_ino,  
               d->d_name);  
    }  
    closedir(dp);  
    return 0;  
}
```

→ opens the current directory

"." → refers to the directory itself

represents one entry in a directory

".." → parent directory

reads one directory entry.

name of file

inode number

each call returns a ptr to struct dirent;
updates the offset;

when no entries are left it returns NULL

Following is the **strace** of **rm** program for removing files.

```
prompt> strace rm foo
...
unlink("foo")           = 0
...
```

→ why not remove or delete?

because of the notion of linking files.
are of two types

(1) Hard links: a new way to make an entry in the file system tree.

→ creates another file that points to the same inode number (and hence same underlying data)

ln <old path name> <new path name>

```
prompt> echo hello > file
prompt> cat file
hello
prompt> ln file file2
prompt> cat file2
hello
```

```
prompt> ls -i file file2
67158084 file
67158084 file2
prompt>
```

- If one file is deleted, the file data can be accessed through the other links.
- inode maintains a link count, and file data deleted only when no further links to it.

```
prompt> rm file
removed 'file'
prompt> cat file2
hello
```

- you can only unlink, OS decides when to delete.

limitations of hard links.

- can't create one to a directory (to avoid cycles in directory tree)
- can't hard link to files in other disk partitions (as inode numbers are unique within a particular file system)

soft links or symbolic links → a third file type

regular file ←

```
prompt> ls -al
drwxr-x--- 2 remzi remzi 29 May 3 19:10 ./
drwxr-x--- 27 remzi remzi 4096 May 3 15:14 ../
-rw-r----- 1 remzi remzi 6 May 3 19:10 file
lrwxrwxrwx 1 remzi remzi 4 May 3 19:10 file2 -> file
```

↳ soft link

holds the path name of the linked-to file as data of the link file.


```
prompt> echo hello > longerfilename
prompt> ln -s longerfilename file3
prompt> ls -al longerfilename file3
-rw-r----- 1 remzi remzi  6 May  3 19:17 longerfilename
lrwxrwxrwx 1 remzi remzi 15 May  3 19:17 file3 ->
longerfilename
```

```
prompt> echo hello > file
prompt> ln -s file file2
prompt> cat file2
hello
```

→ a file that simply stores a pointer to another file name

If the main file is deleted, then the link points to an invalid entry; dangling reference.

```
prompt> echo hello > file
prompt> ln -s file file2
prompt> cat file2
hello
prompt> rm file
prompt> cat file2
cat: file2: No such file or directory
```

Many file systems

Users often want to use many file systems

For eg:- main disk, backup disk, AFS, thumb drives, etc..

What is the most elegant way to support this?

Idea: stitch all the file systems together into a super file system.

mount () system call.

specifies the type of file system (here, ext3)

```
prompt> mount -t ext3 /dev/sda1 /home/users
```

the device file representing the storage partition you want to mount

the mount point

attaches (mounts) a file system so it becomes accessible in the directory tree.

i.e., the directory where the file system will appear.

what actually happens?

Think of linux as having one big directory tree starting from / when you mount a file system:

- you take a device (eg:- disk partition, USB, etc.)
- you attach its root directory to some existing directory (the mount point)
- after that, everything inside the disk appears under that directory.

